

Dynamic Rupture Implementation Plan v2

Date: 2026-04-11 **Status:** Draft — optimized for integration with refactored SEAS-MFEM **Predecessor:** v1 (initial draft — too much new code, insufficient reuse) **Benchmark:** SCEC TPV102 (half-space, aging law rate-and-state friction)

Changes from v1

#	v1 Problem	v2 Resolution
1	FaultRiemann class duplicated friction solver	Reuse DieterichRuinaFriction::SolveSlipRateVectorPsi() — same mathematical form as QD ($\tau - \eta \cdot V = \sigma_n \cdot f(V, \theta)$), just different η and τ sources
2	Standalone WaveOperator class unrelated to existing hierarchy	WaveOperator inherits DomainOperator<MeshType> — FaultGeometry, FaultBasis, RestrictToOwnedFault, ExpandOwnedToLocalFault all work automatically
3	New config system (TPV102Params, tpv102.toml) parallel to existing	Extend SEASConfig with DynamicConfig subsection; use existing bridge pattern (BuildBP5Params → BuildTPV102Params)
4	New output classes for station time series	Reuse ProbeOutput directly — already generic columnar output
5	Separate aging law update function	Reuse AgingLawPsi::Evaluate() from friction/state_evolution.hpp
6	Separate GodunovFlux class with 9×9 matrix algebra	Derive flux from ConstitutiveModel::ComputeTangent() — consistent with Phase 3 design where DG integrators use constitutive interface
7	FaultRiemann had Newton-Raphson solver	Reuse existing Brent solver — proven robust for R&S friction, no new solver needed

Key mathematical insight (eliminates most of Phase 3 from v1):

The QD friction balance: $\sigma_n \cdot f(V, \theta) = \tau_{qs} + \eta \cdot V$ where $\eta = \mu / (2c_s)$

The DR friction balance: $\sigma_n \cdot f(V, \theta) = \theta - \eta_s \cdot V$ where $\eta_s = Z_s^+ \cdot Z_s^- / (Z_s^+ + Z_s^-)$

For homogeneous material: $\eta_s = Z_s / 2 = \rho \cdot c_s / 2 = \mu / (2c_s) = \eta$. **Identical.**

The existing SolveSlipRateVectorPsi(tau, psi, sigma_n, eta, a) works for both regimes — just pass the Riemann trial traction as tau and the impedance as eta.

Overview

Build a 3D dynamic rupture DG code by extending the existing SEAS-MFEM module hierarchy. The WaveOperator inherits from DomainOperator<MeshType> and implements TimeDependentOperator::Mult(),

reusing FaultBasis, FaultGeometry, DieterichRuinaFriction, BoundaryConfig, ConstitutiveModel, and ProbeOutput unchanged. New code is limited to: (1) velocity-stress DG volume/face assembly, (2) Godunov flux derivation from ComputeTangent(), (3) fault-face trial traction and imposed state arithmetic, (4) TPV102-specific initialization.

Estimated new code: ~3,000 LOC (down from ~5,000–8,000 in v1).

Reuse Map

Existing Component	File	Reuse for Dynamic	How
DomainOperator<MeshType>	domain/domain_operator.hpp	Base class for WaveOperator	Inherit; implement Solve(), ComputeTraction(), fault accessors
ConstitutiveModel / LinearElastic	constitutive/linear_elastic.hpp	Material properties, wave speeds, stiffness tensor	ComputeTangent() → Jacobian matrices A, B, C; GetMaxWaveSpeed() → CFL
BoundaryConfig	domain/boundary_config.hpp	Absorbing + free-surface BC classification	natural_attrs → free surface; add absorbing_attrs field
DomainConfig	domain/domain_config.hpp	Solver parameters (face_basis_type, penalty)	Add cfl_factor field for explicit stepping
FaultBasis	fault/fault_basis.hpp	Normal/tangent transforms at fault quad points	Direct call: ProjectTraction(), NormalStress(), EmbedSlipQP()
FaultGeometry	fault/fault_geometry.hpp	Fault ID management, spatial params	Construct from WaveOperator via DomainOperator<MeshType>

Existing Component	File	Reuse for Dynamic	How
DieterichRuinaFriction	friction/dieterich_ruina.cpp	Friction solve: $(\mu, \theta, \sigma_n, \eta, a) \rightarrow V$	Solve SlipRateVectorPsi with Riemann trial traction as τ
AgingLawPsi	friction/state_variable.cpp	State variable evolution $d\psi/dt$	Evaluate(V, psi, ...) for RK integration of ψ alongside Q
SEASConfig	config/seas_config.toml	Configuration + TOML parsing	Extend SimulationConfig with DynamicSettings
ProbeOutput	io/probe_output.cpp	Station time series output	Direct use — already generic columnar I/O
MPIContext	common/mpi_context.cpp	Parallel reductions, communication	Unchanged
FaultScatter	common/fault_ghost_exchange.cpp	Ghost exchange for shared fault faces	Unchanged

Constraints

- **Interface constraints:** DomainOperator, ConstitutiveModel, FaultBasis, FaultGeometry, DieterichRuinaFriction interfaces unchanged. WaveOperator adds to them, never modifies.
- **Dependency constraints:** No new external libraries. Uses MFEM L2_FECollection, ODESolver, ParMesh.
- **Convention constraints:** Follow module-per-directory (dynamic/), test-per-feature (test_wave_*.cpp), TOML config, Makefile integration.
- **Numerical constraints:** CFL-limited explicit stepping. Brent solver convergence $|g| < 10^{-8}$. Energy conservation at fault interface.

Phase 1: WaveOperator Core — Volume Integrals + Interior Flux

Goal

A `WaveOperator<ParMesh>` that inherits `DomainOperator<ParMesh>`, solves the velocity-stress wave equation on a homogeneous cube with Godunov flux at interior faces, verified by plane wave convergence.

Files to Create

- `dynamic/wave_operator.hpp` — Class declaration inheriting `DomainOperator<MeshType>` + `TimeDependentOperator`
- `dynamic/wave_operator.cpp` — Volume integral, face flux, mass inverse
- `dynamic/godunov_flux.hpp` — Upwind flux derived from `ConstitutiveModel::ComputeTangent()`
- `tests/unit/test_wave_operator.cpp` — Plane wave convergence test

Files to Modify

- `domain/boundary_config.hpp` — Add `absorbing_attrs` field (one line)
- `domain/domain_config.hpp` — Add `cfl_factor` field (one line)
- `Makefile` — Add dynamic rupture build targets

Detailed Requirements

1. `WaveOperator<MeshType>` class hierarchy

```
// dynamic/wave_operator.hpp
#include "../domain/domain_operator.hpp"

template <typename MeshType = ParMesh>
class WaveOperator : public DomainOperator<MeshType>,
                    public TimeDependentOperator
{
public:
    static constexpr int NUM_STATE = 9;
    // Q = (σxx, σyy, σzz, σxy, σyz, σxz, vx, vy, vz)

    WaveOperator(MeshType &mesh, int order,
                 const ConstitutiveModel &model,
                 real_t rho,
                 const BoundaryConfig &bdr_config,
                 const DomainConfig &config = {});

    // --- TimeDependentOperator interface ---
    // Compute dQ/dt = M$^{-1}$ · (volume_term + face_flux_term)
    void Mult(const Vector &Q, Vector &dQdt) const override;

    // --- DomainOperator interface (fault access) ---
    int NumComponents() const override { return 3; }
    int Dimension() const override { return 3; }
    int NumSlipComponents() const override { return 2; }
```

```

void Solve(real_t time, const Vector &slip_bc,
           GridFuncType &displacement) override;
void ComputeTraction(const GridFuncType &displacement,
                    const Vector &slip_bc,
                    Vector &traction,
                    Vector *normal_traction = nullptr) override;
FESpaceType &GetFESpace() override;
MeshType &GetMesh() override { return mesh_; }
real_t GetShearModulus() const override;
int GetNumFaultDOFs() const override;
void GetFaultDepths(Vector &depths) const override;
const Array<int> &GetFaultDOFs() const override;
const FaultBasis *GetFaultBasis() const override;
// ... remaining DomainOperator methods (same pattern as ElasticityDomainOperator)

// --- Wave-specific ---
real_t GetMaxDt() const; // CFL-limited time step
real_t GetCp() const { return cp_; }
real_t GetCs() const { return cs_; }

private:
MeshType &mesh_;
int order_;
const ConstitutiveModel *model_;
real_t rho_, lambda_, mu_, cp_, cs_;

std::unique_ptr<L2_FECollection> fec_;
std::unique_ptr<FESpaceType> fes_; // L2, vdim=9

// Precomputed element mass inverse
std::vector<DenseMatrix> elem_mass_inv_;

// Fault infrastructure (shared with QD via DomainOperator)
FaultBasis fault_basis_;
Array<int> fault_interior_faces_, fault_shared_faces_;
Array<int> fault_dofs_;
Vector fault_depths_, coords_x2_, coords_x3_;

// BC classification
BoundaryConfig bdr_config_;
enum class FaceType { Interior, Absorbing, FreeSurface, Fault };

// Assembly methods
void SetupFaultInfo(); // Reuse pattern from ElasticityDomainOperator
void AssembleElementMassInverse();
void ComputeVolumeRHS(const Vector &Q, Vector &rhs) const;
void ComputeFaceFluxRHS(const Vector &Q, Vector &rhs) const;
};

```

Design note: The FE space uses `L2_FECollection(order, 3)` with `vdim = 9` to store all 9 velocity-stress components. Element mass matrix is block-diagonal (DG), precomputed and inverted at construction.

2. Godunov flux from `ComputeTangent()`

The 9×9 Jacobian matrices A , B , C are derived from the 6×6 Voigt stiffness tensor C_{tang} and density ρ . This is consistent with Phase 3's design where DG integrators derive face operations from `ComputeTangent()`:

```
// dynamic/godunov_flux.hpp
class GodunovFlux {
public:
    /// Construct from constitutive model and density.
    /// Extracts  $\lambda$ ,  $\mu$  from ComputeTangent() and computes wave speeds.
    GodunovFlux(const ConstitutiveModel &model, real_t rho);

    /// Interior face flux:  $F^h = \frac{1}{2}T(A+|A|)T^{\top}Q_{\text{self}} + \frac{1}{2}T(A-|A|)T^{\top}Q_{\text{nbr}}$ 
    void Interior(const real_t normal[3],
                 const real_t Q_self[9], const real_t Q_nbr[9],
                 real_t flux[9]) const;

    /// Absorbing:  $F^{\text{abs}} = \frac{1}{2}T(A+|A|)T^{\top}Q_{\text{self}}$ 
    void Absorbing(const real_t normal[3],
                  const real_t Q_self[9],
                  real_t flux[9]) const;

    /// Free surface:  $F^{\text{free}}$  with  $\Gamma$  stress mirror
    void FreeSurface(const real_t normal[3],
                    const real_t Q_self[9],
                    real_t flux[9]) const;

    real_t MaxWaveSpeed() const { return cp_; }

private:
    real_t lambda_, mu_, rho_, cp_, cs_;
    /// Precomputed split flux matrices  $(A+|A|)/2$  and  $(A-|A|)/2$ 
    /// in the x-direction (rotated per face via  $T$  matrix)
    DenseMatrix Aplus_, Aminus_; //  $9 \times 9$ 
};
```

Implementation: For each face, rotate Q to face-normal coordinates via rotation matrix T (built from normal + two tangents), apply A_{plus} to Q_{self} and A_{minus} to Q_{nbr} , rotate back. For homogeneous material, $A_{\text{plus}}/A_{\text{minus}}$ are constant — precompute once.

3. `BoundaryConfig` extension (minimal — one field):

```
// In domain/boundary_config.hpp, add:
struct BoundaryConfig {
    std::set<int> dirichlet_attrs;
    std::set<int> natural_attrs;
```

```

std::set<int> absorbing_attrs; // NEW: for wave equation absorbing BCs
int fault_attr = 3;
// ... rest unchanged
};

```

For QD, absorbing_attrs is empty (ignored). For dynamic, it classifies boundaries.

4. DomainConfig extension (minimal — one field):

```

// In domain/domain_config.hpp, add:
struct DomainConfig {
    // ... existing fields ...
    real_t cfl_factor = 0.5; // NEW: CFL safety factor for explicit stepping
};

```

5. DomainOperator::Solve() in WaveOperator:

Override but with different semantics: for dynamic mode, Solve() is not used by the coupling operator. Override to call MFEM_ABORT("WaveOperator: use Mult() for explicit time stepping") in Phase 1. In Phase 3 (fault integration), it may be repurposed.

6. Fault setup — reuse the pattern from ElasticityDomainOperator::SetupFaultInfo():

- Iterate boundary faces, classify by bdr_config_.fault_attr
- Build fault_interior_faces_, fault_shared_faces_
- Compute FaultBasis via fault_basis_.Compute() and ComputeQPBasis()
- Extract depths and 2D coordinates
- Setup ExpandOwnedToLocalFault / RestrictToOwnedFault communication

Acceptance Criteria

- ☐ WaveOperator compiles, inherits DomainOperator<ParMesh>
- ☐ FaultGeometry<ParMesh> fg(wave_op, params, &mpi) constructs successfully
- ☐ wave_op.GetFaultBasis() returns valid FaultBasis*
- ☐ Plane P-wave in homogeneous cube: speed matches c_p within 1% at 10 elem/wavelength
- ☐ Plane S-wave in homogeneous cube: speed matches c_s within 1%
- ☐ Convergence: L2 error $\propto h^{N+1}$ for order $N = 1, 2$
- ☐ Energy conservation: $|\Delta E/E| < 10^{-12}$ in closed reflecting-wall box
- ☐ make test passes (all existing QD tests unaffected)

Dependencies

- Depends on: DomainOperator (existing), ConstitutiveModel (existing), BoundaryConfig (existing, extend)
- Required by: Phase 2, 3

Phase 2: Boundary Conditions + Free Surface

Goal

Wave operator handles absorbing and free-surface boundaries, verified by surface reflection test.

Files to Modify

- `dynamic/wave_operator.cpp` — BC dispatch in face flux loop (absorbing, free surface)

Files to Create

- `tests/unit/test_wave_bc.cpp` — Reflection + absorption tests

Detailed Requirements

1. Face classification at setup:

- `bdr_config_.absorbing_attrs` → `FaceType::Absorbing`
- `bdr_config_.natural_attrs` → `FaceType::FreeSurface` (traction-free)
- `bdr_config_.fault_attr` matched faces → `FaceType::Fault`
- Interior faces → `FaceType::Interior`

2. In `ComputeFaceFluxRHS()`, dispatch per face type:

```
switch (face_type[f]) {  
    case FaceType::Interior:    flux_.Interior(n, Q_self, Q_nbr, F); break;  
    case FaceType::Absorbing:   flux_.Absorbing(n, Q_self, F); break;  
    case FaceType::FreeSurface: flux_.FreeSurface(n, Q_self, F); break;  
    case FaceType::Fault:       /* Phase 3 */ break;  
}
```

Acceptance Criteria

- ☐ P-wave reflects off free surface with correct $P \rightarrow P$ and $P \rightarrow SV$ conversion coefficients
- ☐ Plane wave exits absorbing boundary with $< 1\%$ reflected energy at normal incidence
- ☐ Corner case: element touching both free surface and absorbing boundary handles correctly
- ☐ Parallel: shared faces at domain boundaries handled correctly

Dependencies

- Depends on: Phase 1
 - Required by: Phase 4 (TPV102 needs free surface + absorbing)
-

Phase 3: Fault-Face Riemann Solver Using Existing Friction Infrastructure

Goal

Dynamic rupture on a fault interface, with the Riemann solver reusing `DieterichRuinaFriction::SolveSlipRateVectorP` for the friction solve and `FaultBasis` for coordinate transforms.

Files to Create

- `dynamic/fault_face_flux.hpp` — Trial traction + imposed state (arithmetic only, ~200 LOC)
- `dynamic/fault_face_flux.cpp` — Implementation
- `tests/unit/test_fault_face_flux.cpp` — Flux unit tests

Files to Modify

- `dynamic/wave_operator.cpp` — Integrate fault flux into face loop

Detailed Requirements

1. FaultFaceFlux — The minimal new code

This class computes ONLY the Riemann-specific arithmetic. The friction solve delegates to existing `DieterichRuinaFriction`.

```
// dynamic/fault_face_flux.hpp
class FaultFaceFlux {
public:
    /// Per-DOF fault data (set at initialization, updated per step)
    struct DOFData {
        real_t invZp_plus, invZp_minus; // Inverse P-impedances
        real_t invZs_plus, invZs_minus; // Inverse S-impedances
        real_t etaP, etaS;              // Harmonic-mean impedances
        real_t sigma_n0, tau1_0, tau2_0; // Initial stress (fault-local)
        real_t a, Dc;                   // Spatially varying friction params
        real_t psi;                     // Current state variable
        real_t slip_rate;               // Current slip rate magnitude (for V guess)
    };

    FaultFaceFlux(DieterichRuinaFriction *friction,
                  const FaultBasis *basis,
                  real_t rho, real_t cp, real_t cs);

    /// Compute fault flux at one quadrature point.
    ///
    /// Steps:
    /// 1. Project Q_plus, Q_minus to fault-local coords via FaultBasis
    /// 2. Compute trial traction (impedance-weighted Riemann average)
    /// 3. Call DieterichRuinaFriction::SolveSlipRateVectorPsi()
    /// 4. Compute imposed state for both sides
    ///
    /// @param fi      Face index (for FaultBasis lookup)
    /// @param qi      Quadrature point index
    /// @param data     Per-DOF fault data (impedances, initial stress, etc.)
    /// @param Q_plus   State on plus side [9]
    /// @param Q_minus  State on minus side [9]
    /// @param dt       Time step (for state variable update)
    /// @param imposed_plus Output: imposed state for plus side [9]
    /// @param imposed_minus Output: imposed state for minus side [9]
    /// @param V_out    Output: slip rate magnitude
    /// @param psi_out  Output: updated state variable
    void Evaluate(int fi, int qi,
                  const DOFData &data,
```

```

        const real_t Q_plus[9],
        const real_t Q_minus[9],
        real_t dt,
        real_t imposed_plus[9],
        real_t imposed_minus[9],
        real_t &V_out,
        real_t &psi_out) const;

private:
    DieterichRuinaFriction *friction_;
    const FaultBasis *basis_;
    real_t Zp_, Zs_; // Impedances (for homogeneous material)
};

```

2. Trial traction computation (Step 2 of Evaluate):

Using FaultBasis::ProjectTraction() and FaultBasis::NormalStress() to rotate from global to fault-local:

```

// Extract fault-local components from global Q states
real_t traction_plus[3] = { Q_plus[SXZ]*n[0] + Q_plus[SXY]*n[1] + Q_plus[SXX]*n[2], ... };
real_t sigma_n_plus = basis_>NormalStress(fi, traction_plus);
real_t tau_local_plus[2];
basis_>ProjectTraction(fi, traction_plus, tau_local_plus);

// Trial (locked-fault Godunov state):
real_t sigma_n_trial = etaP * (v_n_minus - v_n_plus
                               + sigma_n_plus * invZp_plus
                               + sigma_n_minus * invZp_minus);
real_t tau1_trial = etaS * (v_t1_minus - v_t1_plus
                            + tau1_plus * invZs_plus
                            + tau1_minus * invZs_minus);
real_t tau2_trial = etaS * (v_t2_minus - v_t2_plus
                            + tau2_plus * invZs_plus
                            + tau2_minus * invZs_minus);

```

3. Friction solve (Step 3 — reuse existing solver):

```

// Total traction = initial + Riemann trial
real_t tau_total[2] = {
    data.tau1_0 + tau1_trial,
    data.tau2_0 + tau2_trial
};
real_t sigma_n_total = data.sigma_n0 + sigma_n_trial;

// Solve for slip rate using existing Brent solver
// eta parameter = etaS (shear impedance, same role as radiation damping)
real_t V_vec[2];
int iters;
friction_>SolveSlipRateVectorPsi(

```

```
tau_total, data.psi,
std::abs(sigma_n_total), data.etaS, data.a,
V_vec, &iters);
```

This is the critical reuse point. The existing `SolveSlipRateVectorPsi()` handles:

- Brent bracket finding for $|V|$
- Vector decomposition of V into components
- Degenerate-bracket guards for extreme ψ values
- All the numerical stability issues documented in debug v1-v13

4. **Imposed state** (Step 4 — Uphoff eq. 4.60):

```
// Corrected traction after friction
real_t tau1_corrected = tau1_trial - data.etaS * V_vec[0];
real_t tau2_corrected = tau2_trial - data.etaS * V_vec[1];

// Imposed velocity (plus side: + correction, minus side: - correction)
// Plus: v_imposed = v_self + (1/Zs) * (tau_corrected - tau_self)
// Minus: v_imposed = v_self - (1/Zs) * (tau_corrected - tau_self)
```

Then rotate imposed state back to global coordinates via `FaultBasis::EmbedSlipQP()`.

5. **State variable update** — use `AgingLawPsi::Evaluate()` or analytic:

```
// Analytic aging law:  $\theta = \theta_{old} \cdot \exp(-V \cdot dt/L) + (L/V) \cdot (1 - \exp(-V \cdot dt/L))$ 
// In psi-space: use existing AgingLawPsi with V and dt
real_t V_mag = std::sqrt(V_vec[0]*V_vec[0] + V_vec[1]*V_vec[1]);
psi_out = aging_law_.UpdateAnalytic(data.psi, V_mag, data.Dc, dt);
```

Edge Cases

- $V \rightarrow 0$: `SolveSlipRateVectorPsi()` already handles this (degenerate bracket guard from debug v7)
- Tensile σ_n : clamp `sigma_n_total = min(sigma_n_total, 0)` (compression negative)
- Parallel shared fault faces: use existing `FaultScatter` for ghost exchange before `Evaluate`

Acceptance Criteria

- ☐ Locked fault ($\mu \rightarrow \infty$): slip rate = 0, traction = trial traction
- ☐ Frictionless fault ($\mu = 0$): $V = |\tau_{trial}|/\eta_s$, full stress drop
- ☐ `SolveSlipRateVectorPsi()` called with Riemann trial values produces same V as `SeisSol` reference for same inputs
- ☐ State variable θ after 100 steps matches analytic solution (constant V) to 10^{-10}
- ☐ Newton/Brent convergence: < 15 iterations for TPV102 parameters
- ☐ Energy balance: $\int \tau \cdot V \, dA$ = dissipation rate (conservation check)

Dependencies

- Depends on: Phase 1 (wave operator), `DieterichRuinaFriction` (existing), `FaultBasis` (existing), `AgingLawPsi` (existing)
- Required by: Phase 4

Phase 4: TPV102 Full Benchmark

Goal

Complete TPV102 simulation using TOML config + existing bridge pattern, producing SCEC-format output at all stations.

Files to Create

- `config/tpv102_params.hpp` — TPV102 parameter struct (from benchmark spec)
- `config/tpv102.toml` — TOML configuration (extends `SEASConfig`)
- `dynamic/tpv102_setup.hpp` — Initialization: $a(x,y)$, $\theta_{ini}(x,y)$, nucleation perturbation, station definitions
- `drivers/tpv102_driver.cpp` — Main driver (follows `seas_driver.cpp` pattern)
- `tests/verification/tpv102_verification.cpp` — Comparison test

Files to Modify

- `config/seas_config.hpp` — Add `DynamicSettings` to `SimulationConfig`
- `config/seas_config_parser.hpp` — Parse `[simulation.dynamic]` section
- `config/seas_config_bridge.hpp` — Add `BuildTPV102Params()` bridge
- `Makefile` — Add `tpv102` driver and test targets

Detailed Requirements

1. Extend `SEASConfig`:

```
// In config/seas_config.hpp
struct SimulationConfig {
    std::string mode = "qd";    // "qd", "dynamic", "hybrid"

    // Dynamic-only settings (ignored when mode == "qd")
    struct DynamicSettings {
        real_t cfl_factor = 0.5;
        std::string time_integrator = "rk4"; // "rk4" or "rk45"
        int output_interval = 100;           // steps between output
        real_t output_dt = 0.01;             // alternative: fixed dt output
    } dynamic;
};
```

2. TOML config (`config/tpv102.toml`):

```
benchmark = "tpv102"

[mesh]
file = "tpv102/mesh/tpv102_500m.msh"
scale = 1000 # km  $\rightarrow$  m
order = 2
```

```

[material]
density = 2670.0
cs = 3464.0
cp = 6000.0
nu = 0.25

[boundary]
absorbing = [1, 2, 3, 4] # far-field faces
natural = [5]           # free surface (y=0)
fault = 6               # fault plane (z=0)

[solver]
dg_method = "IP"

[time]
t_final = 12.0

[simulation]
mode = "dynamic"

[simulation.dynamic]
cfl_factor = 0.5
time_integrator = "rk4"
output_dt = 0.01

[output]
output_dir = "tpv102_results"
output_prefix = "tpv102"

```

3. **TPV102Params struct** — all values from benchmark spec (see v1 for full table). Bridge function:

```

// In config/seas_config_bridge.hpp
TPV102Params BuildTPV102Params(const SEASConfig &config);

```

4. **TPV102 initialization** (dynamic/tpv102_setup.hpp):

- `real_t ComputeA(real_t x, real_t y, const TPV102Params &p)` — spatially varying $a(x,y)$ with C_∞ boxcar transition
- `real_t ComputeThetaInit(real_t a, const TPV102Params &p)` — self-consistent θ_{ini} from Eq. 6
- `void ApplyNucleation(real_t x, real_t y, real_t t, const TPV102Params &p, real_t &delta_tau)` — stress perturbation $\Delta\tau(r,t)$
- Station definitions: 9 fault stations + 6 surface stations (from benchmark spec)

5. **Driver** (drivers/tpv102_driver.cpp) — follows `seas_driver.cpp` pattern:

- Parse TOML $\rightarrow$ SEASConfig
- Build bridge objects: TPV102Params, LinearElastic, BoundaryConfig
- Construct `WaveOperator<ParMesh>`
- Construct `FaultGeometry<ParMesh>` from `WaveOperator` (tests `DomainOperator` inheritance!)

- Initialize fault DOF data (a , θ_{ini} , initial stress)
- Initialize Q field ($\sigma_{zx} = \tau_{ini}$, $\sigma_{zz} = -\sigma_n$, $v_x = \pm V_{ini}/2$)
- Time loop with RK4: $Q_{new} = Q_{old} + dt \cdot \text{Mult}(Q_{old})$
- Station output via ProbeOutput
- Nucleation perturbation applied each step for $t < T$

6. **Mesh:** Use Gmsh to create half-space domain. Provide .geo script in `tpv102/mesh/`.

Acceptance Criteria

- ☐ Rupture nucleates at hypocenter within $t < 2s$
- ☐ Rupture propagates bilaterally and arrests at VW/VS boundary
- ☐ Peak slip rate at station (0, 7.5km) within 10% of community median
- ☐ Rupture arrival time at station (0, 7.5km) within 5% of community median
- ☐ Free-surface displacement shows correct P and S wave arrivals
- ☐ Simulation completes 12s without instability
- ☐ make test still passes (all QD tests unaffected)

Dependencies

- Depends on: Phase 1, 2, 3
 - Required by: Phase 5
-

Phase 5: QD↔FD Transfer Design

Goal

State conversion between quasi-dynamic (displacement u , slip, ψ) and dynamic (velocity-stress Q , slip, ψ), enabling future hybrid simulations.

Files to Create

- `dynamic/regime_transfer.hpp` — Conversion utilities + snapshot protocol
- `solver/seas_hybrid_operator.hpp` — Three-mode dispatcher skeleton
- `tests/unit/test_regime_transfer.cpp` — Round-trip conversion test

Detailed Requirements

1. Displacement → Velocity-Stress conversion:

```
// In dynamic/regime_transfer.hpp
namespace RegimeTransfer {

    /// Convert QD displacement field to velocity-stress Q.
    ///  $\sigma = C:\nabla u$  (element-local via ConstitutiveModel::ComputeStress)
    ///  $v = 0$  (or dip from last QD slip rate)
    void QDtoFD(const DomainOperator<ParMesh> &qd_domain,
               const ParGridFunction &displacement,
```

```

        const ConstitutiveModel &model,
        WaveOperator<ParMesh> &wave_op,
        Vector &Q_initial);

    /// Integrate fault slip accumulated during FD phase.
    /// slip_new = slip_frozen + $\int V dt$ (from FaultFaceFlux records)
    void AccumulateFDSlip(const Vector &slip_frozen,
                        const Vector &fd_slip_increment,
                        Vector &slip_new);

    /// Restore QD state after FD phase.
    /// Performs one QD Solve() with updated slip BC for consistency.
    void FDtoQD(const Vector &fd_slip_new,
               const Vector &fd_psi_new,
               SEASQuasiDynamicOperator<ParMesh> &qd_op,
               Vector &qd_state);

} // namespace RegimeTransfer

```

2. Hybrid operator skeleton:

```

/// solver/seas_hybrid_operator.hpp
template <typename MeshType = ParMesh>
class SEASHybridOperator : public TimeDependentOperator {
public:
    enum class Regime { QD, FD };

    SEASHybridOperator(
        SEASQuasiDynamicOperator<MeshType> *qd_op,
        WaveOperator<MeshType> *wave_op,
        real_t V_activate = 1e-3,
        real_t V_deactivate = 1e-6);

    void Mult(const Vector &state, Vector &rate) const override;
    Regime GetRegime() const { return regime_; }

private:
    Regime regime_ = Regime::QD;
    /// Actual switching logic deferred to future work
};

```

Acceptance Criteria

- ☐ QDtoFD(): stress field matches QD traction at fault DOFs to 10^{-12}
- ☐ Round-trip QD→FD→QD with zero FD evolution: state matches to 10^{-14}
- ☐ Fault slip and ψ preserved across transition
- ☐ SEASHybridOperator compiles and dispatches to QD mode

Dependencies

- Depends on: Phase 4 (proven FD code), existing QD infrastructure
 - Required by: future hybrid work
-

Phase 6: GPU Acceleration Exploration

Goal

Profile CPU dynamic rupture, implement GPU-accelerated kernels for volume + face integrals using MFEM device infrastructure.

Files to Create

- `dynamic/wave_kernels_gpu.hpp` — GPU kernels using MFEM_FORALL
- `tests/unit/test_wave_gpu.cpp` — GPU vs CPU equivalence test

Detailed Requirements

1. Use MFEM device infrastructure:

- `MFEM_FORALL(i, N, { ... })` for element/face loops
- `MFEM_HOST_DEVICE` lambdas for portable kernels
- `Device::Configure("cuda")` at startup
- Memory management via `Vector::UseDevice(true)`

2. Target kernels (from profiling):

- Element mass inverse application (embarrassingly parallel)
- Volume integral (element-local)
- Interior face Godunov flux (face-local)
- Fault-face friction solve (DOF-local — most complex)

3. Pattern follows MFEM ex9 (DG advection with PA):

```
// Element-local volume integral
auto d_Q = Q.Read();    // Device pointer
auto d_rhs = rhs.ReadWrite();
MFEM_FORALL(e, num_elements, {
    // Load element DOFs
    // Compute flux at quadrature points
    // Contract with basis gradients
    // Accumulate into rhs
});
```

Acceptance Criteria

- ☐ CPU profiling report documents bottleneck breakdown
- ☐ GPU produces identical results to CPU ($L_2 < 10^{-12}$)
- ☐ 5× speedup over single-core CPU for TPV102

- Works with Device("cpu") when no GPU available

Dependencies

- Depends on: Phase 4 (working CPU code)
- Required by: nothing (optimization)

Testing Strategy

Phase	Test	Verifies	Method
1	test_wave_operator	Wave speed, convergence order	Plane wave in cube
1	test_wave_operator	FaultGeometry construction	Construct from WaveOperator
1	test_wave_operator	Energy conservation	Closed reflecting box
2	test_wave_bc	Free surface reflection	P-wave hitting y=0
2	test_wave_bc	Absorbing boundary	Plane wave exits domain
3	test_fault_face_flu	Locked/frictionless fault	Limit cases
3	test_fault_face_flu	Friction solver reuse	Compare V with QD solver at same inputs
4	tpv102_verification	Full benchmark	Community comparison
5	test_regime_transition	Round-trip conversion	QD→FD→QD identity
6	test_wave_gpu	GPU equivalence	CPU vs GPU L2

Risk Assessment

Risk	Likelihood	Impact	Mitigation
SolveSlipRateVecMedium(η parameter mismatch)	Medium	High	Unit test: call with Riemann η_s , compare V against SeisSol at identical inputs
WaveOperator DomainOperator inheritance breaks	Low	High	Phase 1 acceptance criterion: FaultGeometry(wave_op, params) constructs
FaultGeometry Sign convention mismatch at fault (QD vs DR normals)	Medium	High	Unit test: locked fault produces zero slip; wrong sign → immediate blowup

Risk	Likelihood	Impact	Mitigation
L2 9-component FE space memory overhead	Medium	Medium	Profile; consider separate scalar spaces if >2× overhead
DG mass matrix non-diagonal for deformed elements	Low	Medium	Precompute dense inverse per element ($O(ndof^3)$, $ndof$ small)

Project Layout After All Phases

miniapps/seas/	
├─ dynamic/	NEW (Phases 1–6)
│ └─ wave_operator.hpp	Phase 1: inherits DomainOperator
│ └─ wave_operator.cpp	Phase 1: volume + face assembly
│ └─ godunov_flux.hpp	Phase 1: upwind flux from ComputeTangent
│ └─ fault_face_flux.hpp	Phase 3: trial traction + imposed state (~200 LOC)
│ └─ fault_face_flux.cpp	Phase 3: reuses DieterichRuinaFriction
│ └─ tpv102_setup.hpp	Phase 4: spatial params, nucleation, stations
│ └─ regime_transfer.hpp	Phase 5: QD $\rightarrow$ FD conversion
│ └─ wave_kernels_gpu.hpp	Phase 6: MFEM_FORALL kernels
├─ config/	
│ └─ tpv102_params.hpp	Phase 4: benchmark parameters
│ └─ tpv102.toml	Phase 4: TOML configuration
├─ domain/	
│ └─ boundary_config.hpp	Phase 1: +absorbing_attrs field
│ └─ domain_config.hpp	Phase 1: +cfl_factor field
├─ drivers/	
│ └─ tpv102_driver.cpp	Phase 4: standalone dynamic driver
├─ solver/	
│ └─ seas_hybrid_operator.hpp	Phase 5: three-mode skeleton
└─ tests/	
│ └─ unit/	
│ │ └─ test_wave_operator.cpp	Phase 1: plane wave, convergence, energy
│ │ └─ test_wave_bc.cpp	Phase 2: surface + absorbing
│ │ └─ test_fault_face_flux.cpp	Phase 3: locked, frictionless, energy
│ │ └─ test_regime_transfer.cpp	Phase 5: round-trip conversion
│ │ └─ test_wave_gpu.cpp	Phase 6: GPU equivalence
│ └─ verification/	
│ │ └─ tpv102_verification.cpp	Phase 4: SCEC benchmark

New files: ~15. New LOC estimate: ~3,000 (vs v1's ~5,000–8,000). **Existing code reused unchanged: ~8,000 LOC** (friction, fault basis, config, I/O).